SPinSynth-T Architecture Description

Initial architecture note for the Teensy 3.1/3.2 monophonic virtual analog synthesizer

Purpose

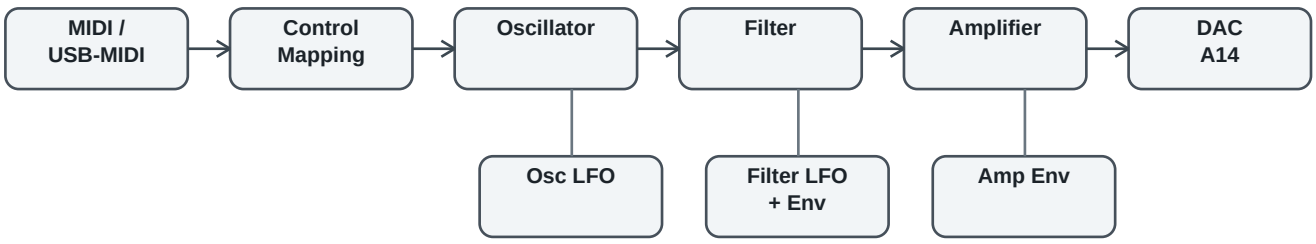
SPinSynth-T is a monophonic virtual analog synthesizer developed by Ricardo Peculis for Teensy 3.1/3.2. The instrument implements its own oscillator, filter, amplifier, envelope generators, low-frequency oscillators, fixed-point helpers, double-buffered audio flow, and MIDI control mapping.

SPinSynth-T does not use the PJRC Teensy Audio Library for audio synthesis. It does, however, use PJRC MIDI and USB-MIDI support.

This document describes the current cleaned architecture and provides a starting point for future refactoring into an SPinSynthT voice class suitable for HMI control and later polyphony.

The current hardware architecture uses the 12-bit DAC available on Teensy 3.1/3.2 as the audio output path. SPinSynth-T does not include its own human-machine interface; performance and sound parameters are received through MIDI notes, pitch bend, and MIDI Control Change messages.

High-Level Signal Flow



- MIDI and USB-MIDI callbacks translate note, pitch bend, and control-change messages into synth parameters.
- The oscillator renders an audio buffer, the filter processes that buffer, and the amplifier applies the amp envelope and volume.
- A timer-driven DAC callback outputs one sample at a time from the amplifier buffer while the control/update timer prepares the next buffers.

Core Modules

Module	Role	Important Features
SPinSynth-T.ino	Hardware shell and control coordinator.	MIDI callbacks, CC mapping, timers, DAC output, buffer phase control.
Oscillator	Primary sound generator.	Sine, saw, square/pulse, XTREME; PolyBLEP saw edges; pulse from two corrected saws; LFO, pitch bend, tuning, portamento.
Filter	Virtual analog low-pass filter stage.	Cutoff/resonance smoothing, LFO cutoff modulation, envelope cutoff modulation, fixed-point buffer processing.
Amplifier	Amplitude stage after the filter.	Amp envelope, velocity scaling, master volume, fixed-point buffer processing.

EnvelopeGenerator	ADSR envelope engine.	Exponential attack/decay/release constants, gate/trigger model, multi-note on-count support, LED gate feedback.
LowFrequencyOscillator	Modulation oscillator.	Sine, saw, inverted saw, square/pulse; pulse width support; used by oscillator and filter.
SynthUtilities	Shared utility layer.	FixedPoint helpers, KeyEvent state, DoubleBuffer audio buffering.

Audio Timing And Buffers

- The active audio rate is 44.1 kHz, with a buffer size of 128 samples.
- The DAC timer calls outputBufferedDAC() at audio rate and writes to the Teensy 3.1/3.2 12-bit DAC output pin A14.
- Three DoubleBuffer instances carry oscillator, filter, and amplifier data.
- The update phase alternates between generating oscillator samples and processing filter/amplifier buffers.
- DoubleBuffer::read() now returns the current sample before advancing, preserving sample zero after each swap.

Oscillator Architecture

The oscillator is the main identity of SPinSynth-T. It combines wavetable lookup, MIDI pitch tracking, pitch bend, master tuning, portamento, oscillator LFO, and multiple waveform modes.

- SINE reads from a sine wavetable and provides the clean reference waveform.
- SAW reads from a saw wavetable through getSawSample(), which applies PolyBLEP correction around the discontinuity.
- SQUARE_PULSE is generated by subtracting two PolyBLEP-corrected saws; the second phase controls pulse width.
- XTREME mixes main saw, detuned saw, sub saw, and pulse components, each controlled by MIDI CC amount/factor parameters.
- getNaiveSample() and updateNaiveBuffer() are intentionally retained as historical comparison paths without PolyBLEP.

Waveform	Generation Method	SPinSynth-T Character
SINE	Direct lookup from the sine wavetable.	Clean tone and reference waveform for testing.
SAW	Saw wavetable sample corrected by getSawSample().	Main bright virtual analog source; PolyBLEP softens the discontinuity.
SQUARE_PULSE	Difference between two corrected saw samples separated by the pulse-width phase offset.	Variable-width pulse without a raw threshold edge; both edges receive PolyBLEP correction.
XTREME	Mix of main saw, detuned saw, sub saw, and pulse derived from the main saw pair.	The richest SPinSynth-T oscillator mode, with MIDI-controlled saw, pulse, sub, detune, and sub-factor parameters.

Pulse And XTREME Generation

The pulse waveform is not produced as a simple high/low comparator. Instead, SPinSynth-T computes two saw samples at different phases and subtracts them. The second phase is offset by the pulse-width control, so moving the pulse-width parameter moves one edge of the pulse. Because each saw sample passes through getSawSample(), both pulse edges benefit from the same PolyBLEP discontinuity correction used by the saw waveform.

XTREME extends this idea by running three saw-related phases: the main oscillator phase, an off-tuned phase, and a sub-oscillator phase. The output mix combines main/detuned saw energy, a pulse component derived from the main saw pair, and a sub component. Separate MIDI controls adjust saw amount, pulse amount, sub amount, detune factor, pulse width, and sub factor. This makes XTREME a compact paraphonic-flavored oscillator mode inside the otherwise monophonic synth.

PolyBLEP Use

PolyBLEP is used at the discontinuities of the saw-derived oscillator paths. In getSawSample(), the oscillator first reads the normal saw wavetable sample. When the phase is close to the start or end of the cycle, the code applies a small polynomial correction. This smooths the abrupt jump that would otherwise create strong aliasing components at higher pitches.

The method is especially effective here because SAW, SQUARE_PULSE, and XTREME all reuse the same corrected saw generator. It is not a complete universal band-limiting solution, but it is a practical and efficient anti-aliasing technique for the Teensy 3.x audio interrupt path. The retained naive oscillator path provides a useful comparison for hearing or viewing the improvement on an oscilloscope.

Filter, Envelope, And Amplifier

- The filter is a four-stage virtual analog style low-pass with resonance feedback and stability limits on cutoff/resonance.
- Filter cutoff can be controlled directly, modulated by the modulation wheel, modulated by an LFO, and shaped by a filter envelope.
- The main cutoff control uses an exponential response for finer low-frequency adjustment.
- The filter envelope amount is bipolar, preserving inverted and normal envelope control with a curved response near the zero-envelope center.
- The envelope generator uses exponential coefficient calculations inspired by analog-style ADSR behavior.
- The amplifier uses its own envelope and combines envelope level, velocity, and master volume in the final buffer stage.

MIDI And Control Model

The current sketch maps Arturia KeyLab-style MIDI controls directly to synth parameters. SPinSynth-T has no built-in HMI, so control parameters are received through MIDI CC messages. The original hardware/software testing used an Arturia KEYLAB25 MIDI controller. Future HMI work should place a parameter layer between input devices and the synth engine.

- Note on/off drives oscillator key-event tracking and both filter/amplifier envelopes.
- Pitch bend is applied in the oscillator frequency path.
- Mod wheel can be assigned to filter cutoff, filter resonance, filter LFO amount, or oscillator LFO amount.
- Control changes set oscillator waveform, LFO parameters, filter ADSR, amplifier ADSR, portamento, tuning, and XTREME component amounts.

Current Architectural Strengths

- The synth engine is handmade and understandable: oscillator, filter, amplifier, envelopes, and LFOs are all project-owned code.
- The oscillator has a distinctive identity, especially through XTREME and the PolyBLEP pulse-from-saw approach.
- The monophonic key-event model already resembles the logic needed for a future voice allocator.
- The recent cleanup has reduced debug debris, removed unused alternate APIs, and clarified the current signal path.

Known Limitations And Cleanup Targets

- Global instances in the sketch should eventually become members of an SPinSynthT class.
- The sketch still mixes hardware, MIDI mapping, parameter interpretation, and audio scheduling responsibilities.
- Some headers still contain historical fixed-point/table-BLEP comments that should be reviewed carefully, not mass-removed.
- Some methods accept DoubleBuffer by value; changing this previously affected behavior, so that area should remain conservative.
- The current output is Teensy 3.x DAC-oriented; a future Teensy 4.x target will need a different output strategy.

Proposed Next Architecture: SPinSynthT Class

The next major refactor should introduce a single class representing one complete synth voice. For the monophonic HMI project, one instance is enough. For a future polyphonic instrument, the same class can become the per-voice unit managed by a higher-level allocator.

- `begin(sampleRate, bufferSize)`
- `noteOn(pitch, velocity)`
- `noteOff(pitch, velocity)`
- `pitchBend(bend)`
- `controlChange(number, value) or setParameter(id, value)`
- `updateControl()`
- `render(outputBuffer)`

Recommended Future Split

- SPinSynthT owns Oscillator, Filter, Amplifier, envelopes, LFOs, and per-voice state.
- The sketch or application shell owns MIDI, HMI scanning, LCD updates, presets, timers, and audio output.
- A Parameter Manager translates MIDI, encoders, presets, and future automation into the same parameter model.
- A future VoiceAllocator can own several SPinSynthT instances for polyphony.

Prepared as an initial architecture note during the SPinSynth-T cleanup collaboration. This is intentionally a living document and should evolve as the SPinSynthT class emerges.